



ETEC RUBENS DE FARIA E SOUZA
Desenvolvimento de Sistemas - Módulo 01
DESENVOLVIMENTO DE SISTEMAS I

Nome Do Aluno: Maria Vitória Moraes

Professor: Paulo Eduardo Cardoso Andrade

RELATÓRIO TÉCNICO: ANÁLISE E REPRESENTAÇÃO VISUAL DE ESTRUTURAS DE LISTA EM PYTHON

Maria Vitória Moraes

RELATÓRIO TÉCNICO:ANÁLISE E REPRESENTAÇÃO VISUAL DE ESTRUTURAS DE LISTA EM PYTHON

Trabalho apresentado como requisito para
avaliação na disciplina Desenvolvimento de
Sistemas I ,do curso de Desenvolvimento de
Sistemas da ETEC Sorocaba.
Professor: Paulo Eduardo Cardoso Andrade.

**Sorocaba - SP
2026**

RESUMO

Este relatório técnico apresenta a análise lógica e a representação visual do funcionamento de um algoritmo desenvolvido na linguagem de programação Python, focado no gerenciamento de rotinas pessoais por meio de estruturas de dados do tipo lista. A abordagem fundamenta-se no estudo do programa “Meu Painel de Dados”, detalhando o fluxo de processamento dividido em quatro etapas: recolha de entradas de dados do utilizador, inicialização e preenchimento dinâmico de vetores com o método `.append()`, varredura e exibição de elementos através do laço `for` e tomada de decisão condicional utilizando a função nativa `len()`. A validação prática do algoritmo foi documentada com capturas de ecrã do terminal e com a elaboração de um fluxograma lógico detalhado. Os resultados comprovam que o agrupamento estruturado de dados simplifica a manipulação de informações e automatiza respostas personalizadas com base no volume de registos. Conclui-se que a modelação visual do fluxo de execução é uma ferramenta eficaz para a consolidação de conceitos fundamentais de lógica de programação e vetores dinâmicos.

Palavras-chave: Python. Estruturas de dados. Listas. Lógica de programação. Fluxograma.

Sumário

1 INTRODUÇÃO	5
2 DESENVOLVIMENTO	5
2.1 Entrada de dados	5
2.2 Criação e preenchimento da lista	6
2.3 Exibição das tarefas cadastradas.....	7
2.4 Decisão final do programa	8
3 REPRESENTAÇÃO VISUAL DO FUNCIONAMENTO	9
4 CONSIDERAÇÕES FINAIS.....	9
REFERÊNCIAS	11

Lista De Ilustrações

Figura 1 — Trecho do código Python para coleta de entrada de dados	6
Figura 2 — Trecho do código Python para inicialização da lista vazia	6
Figura 3 — Trecho do código Python com estrutura de repetição e método append()	6
Figura 4 — Trecho do código Python para varredura da lista e impressão das saídas	6
Figura 5 — Trecho do código Python com estruturas condicionais e verificação por len().....	7
Figura 6 — Exemplo prático de execução do programa e interação no terminal	7
Figura 7 — Fluxograma lógico completo do programa “Meu Painel de Dados”	9

Lista De Tabelas

<u>Tabela</u> 1 — Relação entre a quantidade de tarefas, a condição lógica e a mensagem	7
---	---

1 INTRODUÇÃO

Este trabalho apresenta a análise de um programa desenvolvido na linguagem Python. O algoritmo tem como finalidade cadastrar tarefas da rotina de um usuário, armazená-las em uma lista e apresentar uma mensagem de acordo com a quantidade de tarefas registradas.

A atividade permite compreender como as listas funcionam na programação. Uma lista é uma estrutura utilizada para reunir vários dados relacionados em um único lugar. No programa analisado, ela é usada para guardar as tarefas informadas pelo usuário, como estudar, fazer exercícios, realizar trabalhos ou organizar compromissos.

Além do uso de listas, o algoritmo trabalha com entrada de dados, estruturas de repetição e estruturas condicionais. Dessa forma, o programa recebe informações, organiza os dados cadastrados, percorre os itens armazenados e toma uma decisão com base na quantidade de tarefas existentes.

2 DESENVOLVIMENTO

2.1 Entrada de dados

No início da execução, o programa solicita algumas informações ao usuário. Os primeiros dados pedidos são o nome e a cidade onde a pessoa mora.

Essas informações são armazenadas nas variáveis nome e cidade. As duas variáveis recebem dados do tipo texto, também chamado de *string* em Python.

Em seguida, o programa pergunta quantas tarefas serão cadastradas. Essa informação é armazenada na variável qtd. Como a quantidade representa um número, o valor digitado é convertido para o tipo inteiro por meio da função `int()`.

Os dados recebidos pelo programa são:

- Nome do usuário;
- Cidade do usuário;
- Quantidade de tarefas a serem cadastradas;
- Nome de cada tarefa da rotina.

Como exemplo de personalização do programa, foram utilizados os seguintes dados:

Figura 1 — Trecho do código Python para coleta de entrada de dados

```
1  # =====
2  # 1. ENTRADAS (Coleta de dados)
3  # =====
4  nome = input("Digite seu nome: ")
5  cidade = input("Digite sua cidade: ")
6  qtd = int(input("Quantas tarefas você vai cadastrar? "))
7
```

Fonte: Captura de tela realizada pela autora (2026).

2.2 Criação e preenchimento da lista

Após receber os dados iniciais, o programa cria uma lista vazia chamada tarefas.

Figura 2 — Trecho do código Python para inicialização da lista vazia

```
8  # =====
9  # 2. PROCESSAMENTO (Lógica e Estruturação)
10 # =====
11 tarefas = []
12
```

Fonte: Captura de tela realizada pela autora (2026).

Nesse momento, a lista ainda não possui nenhum elemento. Ela funciona como uma caixa vazia que será preenchida conforme o usuário informar suas tarefas.

Para cadastrar os itens, o programa utiliza uma estrutura de repetição:

Figura 3 — Trecho do código Python com estrutura de repetição e método append()

```
13 # Coleta e processamento da lista de tarefas
14 for i in range(qtd):
15     tarefa = input(f"Digite a tarefa {i+1}: ")
16     tarefas.append(tarefa)
17
```

Fonte: Captura de tela realizada pela autora (2026).

O comando for faz com que o cadastro seja repetido conforme a quantidade definida pelo usuário. Se forem informadas três tarefas, por exemplo, o processo de cadastro será realizado três vezes.

A cada repetição, uma tarefa é digitada e adicionada ao final da lista com o método append().

Figura 4 — Trecho do código Python com estruturas condicionais e verificação por len()

```
18 # Avaliação do status da rotina
19 v if len(tarefas) == 0:
20     status_rotina = "Nenhuma tarefa cadastrada."
21 v elif len(tarefas) <= 3:
22     status_rotina = f"{nome} de {cidade}: rotina leve hoje!"
23 v else:
24     status_rotina = f"{nome} de {cidade}: rotina cheia! Organize prioridades."
25
```

Fonte: Captura de tela realizada pela autora (2026).

2.3 Exibição das tarefas cadastradas

Depois de preencher a lista, o programa percorre os itens cadastrados para mostrá-los na tela.

Figura 5 — Trecho do código Python para varredura da lista e impressão das saídas

```
26 # =====
27 # 3. SAÍDAS (Exibição dos resultados)
28 # =====
29 print("\nTarefas cadastradas:")
30 for t in tarefas:
31     print("-", t)
32
33 print(f"\n{status_rotina}")
```

Fonte: Captura de tela realizada pela autora (2026).

Nessa parte do algoritmo, a variável *t* representa temporariamente cada tarefa presente na lista. O laço *for* passa por todos os itens, um de cada vez, até que não existam mais tarefas para exibir.

Considerando o exemplo anterior, a saída seria:

Figura 6 — Exemplo prático de execução do programa e interação no terminal

```
TERMINAL  PROBLEMAS 1  VISUALIZAR

Digite seu nome: Maria Vitória Moraes
Digite sua cidade: Sorocaba-SP
Quantas tarefas você vai cadastrar? 3
Digite a tarefa 1: aula de Pythoy
Digite a tarefa 2: aula de inglês
> Digite a tarefa 3: trabalho de desenvolvimento

Tarefas cadastradas:
- apresentação de Pythoy
- aula de inglês
- trabalho de desen

Maria Vitoria Moraes de Sorocaba-Sp: rotina leve hoje!

** Processo encerrado - Código de retorno: 0 **
```

Fonte: Captura de terminal realizada pela autora (2026).

Essa etapa demonstra que uma lista permite armazenar vários dados e acessá-los posteriormente de forma organizada.

2.4 Decisão final do programa

Após exibir as tarefas, o programa verifica o tamanho da lista usando a função `len(tarefas)`.

A função `len()` retorna a quantidade de elementos armazenados na lista. A partir desse resultado, o programa escolhe qual mensagem final será apresentada ao usuário.

A lógica utilizada é a seguinte:

Tabela 1 — Relação entre a quantidade de tarefas, a condição lógica e a mensagem final exibida

Quantidade de tarefas	Condição no programa	Mensagem exibida
Nenhuma tarefa	<code>len(tarefas) == 0</code>	"Nenhuma tarefa cadastrada."
De uma a três tarefas	<code>len(tarefas) <= 3</code>	"Maria Vitória Moraes de Sorocaba – SP: rotina leve hoje!"
Mais de três tarefas	<code>else</code>	"Maria Vitória Moraes de Sorocaba – SP: rotina cheia! Organize prioridades."

Fonte: Elaborado pela autora (2026).

A primeira condição verifica se a lista está vazia. Caso não exista nenhuma tarefa cadastrada, o programa informa que não há registros.

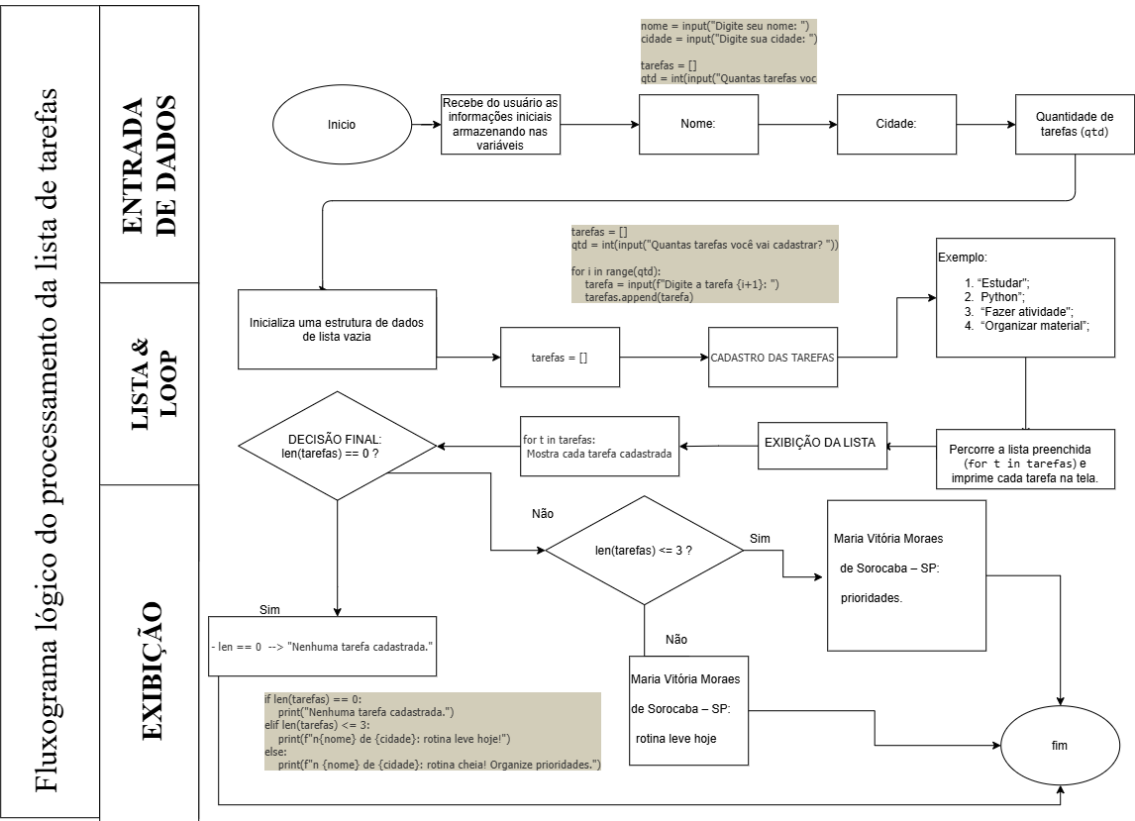
A segunda condição verifica se a lista possui até três tarefas. Nessa situação, o programa considera que o usuário possui uma rotina leve.

Caso a lista tenha mais de três elementos, a estrutura else é executada. Nesse caso, o sistema informa que a rotina está cheia e sugere a organização das prioridades.

3 REPRESENTAÇÃO VISUAL DO FUNCIONAMENTO

A Figura 1 apresenta o fluxo de funcionamento do programa, desde a entrada dos dados até a exibição da mensagem final.

Figura 1 — Fluxo lógico do programa “Meu Paine de Dados”



Fonte: Elaborado por Maria Vitória Moraes (2026).

4 CONSIDERAÇÕES FINAIS

A análise do programa permitiu compreender a importância das listas na linguagem Python. Essa estrutura de dados é utilizada para reunir informações relacionadas, facilitando o armazenamento, a organização e a consulta dos dados inseridos pelo usuário.

No algoritmo estudado, a lista tarefas começa vazia e é preenchida gradualmente com o método `append()`. Em seguida, a estrutura de repetição `for` percorre todos os itens cadastrados e os apresenta na tela de forma organizada.

A função `len()` também possui um papel importante, pois permite verificar a quantidade de elementos armazenados. Com base nesse resultado, o programa decide se o usuário não possui tarefas, possui uma rotina leve ou apresenta uma rotina cheia.

Dessa forma, a atividade demonstra um fluxo básico e importante da programação: receber dados, armazenar informações, processar os elementos cadastrados e apresentar uma resposta de acordo com uma condição definida.

REFERÊNCIAS

MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. *Algoritmos: lógica para desenvolvimento de programação de computadores*. 28. ed. São Paulo: Érica, 2019.

PYTHON SOFTWARE FOUNDATION. *Python documentation: data structures*. [S. l.], [s. d.]. Disponível em: <<https://docs.python.org/3/tutorial/datastructures.html>>. Acesso em: 25 set. 2026.